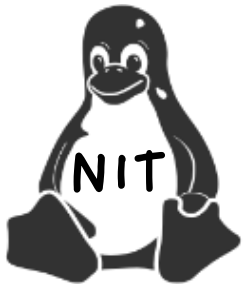


Welcome



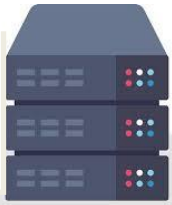
Everyone



MASTERING LINUX!

Day 30

03



Understanding Login Process, PID, and Bash Initialization Files

NIT ACADEMY

- ☐ What is the first Process after login?
- ☐ How to identify the shell PID?
- ☐ Difference between Global and User initialization files.
- ☐ Difference between `.bash_profile` and `.bashrc`.
- ☐ How logout files work (`.bash_logout`).

LOGIN TO LINUX

Important Distinction

There are actually 4 shell types:

Shell Type	Login	Interactive
SSH Login	Yes	Yes
Console Login	Yes	Yes
bash	No	Yes
bash script.sh	No	No

LOGIN INTERACTIVE

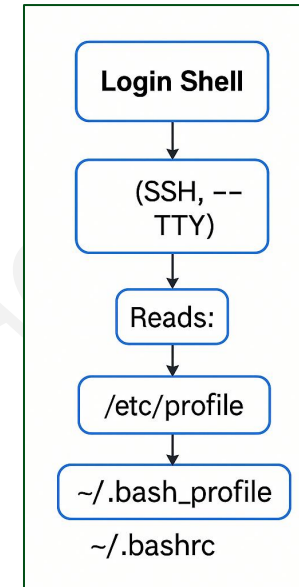
A login shell is created when Linux authenticates you (asks for a password, SSH key, console login, etc.).

```
Rocky Linux 9.5 (Blue Onyx)
Kernel 5.14.0-503.14.1.el9_5.x86_64 on an x86_64

Linux login: root
Password:
```

Login Shell:

- A shell session started **when you log in** to the system (e.g., via SSH, virtual terminal Ctrl+Alt+F1, or console login).
- A **login shell is the first process** that executes after a user successfully logs into a system (either locally or remotely via SSH).
- It **reads configuration files**:
 - `/etc/profile`,
 - `~/.bash_profile`,
 - `~/.profile`.



Feature	Login Shell
Started via	SSH, <code>su -</code> , Console
Reads	<code>/etc/profile</code> , <code>~/.bash_profile</code> , <code>~/.bashrc</code>
Typical use case	User session on login
Example command	<code>su - username</code>

LOGIN – LINUX SHELL

Let us login as a **root** user?

Rocky Linux 9.5 (Blue Onyx)

Kernel 5.14.0-503.14.1.el9_5.x86_64 on an x86_64

Linux login:

Password:

Last login: Sat Jun 6 02:11:01 on tty1

[**root**@Linux ~]#

Run

[**root**@Linux ~]# whoami

Root

How do we know this is a “Login” shell and it is “Interactive” – it accepts commands?

[**root**@Linux ~]# shopt login_shell

Login_shell **on**

LOGIN PROCESS (PID,PPID)

In Linux, processes don't just appear out of nowhere; with very few exceptions, every process is "forked" (spawned) by an existing **parent process**. Here is exactly what hierarchy means step-by-step:

1. systemd (The Root Parent)

What it is: This is the mother of all processes on a modern Linux system, typically holding **PID 1**.

Its role: When your Linux kernel finishes loading, it starts **systemd**. From there, systemd is responsible for bringing up the rest of the operating system, including background services, networking, and graphical interfaces.



2. sshd (The Child of systemd / Parent of bash)

What it is: The Secure Shell Daemon.

Its role: systemd started the sshd service so the server could listen for incoming network connections. When you initiated a connection to this server (whether via standard SSH or a web-based jump portal), sshd caught that request, authenticated your credentials, and opened a secure session for you. **(This becomes the Parent Process ID = PPID)**



3. bash (The Child of sshd)

What it is: Your interactive command-line shell.

Its role: Once sshd verified who you were, it needed to give you an interface to interact with the OS. It spawned bash (which becomes the **PID**).



4. Bash reads and **runs initialization files** => /etc/profile, ~/.bash_profile, ~/.bashrc



5. **SUCCESS:** bash presents the **"[root@Linux ~]"** prompt!

BEFORE LOGIN – SYSTEM (PID1)

What process starts before LOGIN?

[root@Linux ~]# ps 1

PID	TTY	STAT	TIME	COMMAND
-----	-----	------	------	---------

1	?	Ss	0:16	/usr/lib/systemd/systemd --switched-root --system --deserialize 31
---	---	----	------	--

WHAT IS THE FIRST PROCESS AFTER LOGIN

What Process Starts When I login as a **root** user?

\$\$ = \$\$ is a special shell variable that expands to the PID of your current shell

instance[root@Linux ~]# ps **\$\$**

PID	TTY	STAT	TIME	COMMAND
230948	pts/0	Ss	0:00	-bash

[root@Linux ~]# echo **\$\$**

230948

[root@Linux ~]# echo **\$0**

-bash

#notice the leading - .

There is something else on your Linux system that started FIRST before Bash

[root@Linux ~]# ps -fp **\$PPID**

UID	PID	PPID	C	STIME	TTY	TIME	CMD
root	230948	232215	0	01:57	?	00:00:00	sshd: root@pts/0

- PID (**230948**) is the identity of your current running Bash shell.
- PPID (**230943**) is the identity of the process that launched your Bash shell.

HOW TO IDENTIFY THE SHELL PID

We can see the tree structure on our terminal by running “pstree” command. To filter it down to just your current session, you can pass your shell’s parent PID to it so that you will get a visual output showing how sshd is holding onto your bash session, and if you run a command right now (like ps or ls), you will see that command briefly as a child under bash:

```
[root@Linux ~]# ps -fp $$
```

UID	PID	PPID	C	STIME	TTY	TIME	CMD
root	232219	232215	0	03:36	?	00:00:00	sshd: root@pts/0

```
[root@Linux ~]# pstree -p 232215
```

```
sshd(232215)——sshd(232219)——bash(232220)——pstree(233430)
```

WHAT FILES EXECUTE DURING LOGIN?

BASH STARTUP SCRIPTS

INITIALIZATION PROCESS

System Files (Global)

When Root logs in, SYSTEM files run. USER cannot change these:

1. /etc/profile
2. /etc/bashrc

User Files (Local)

User can change these files:

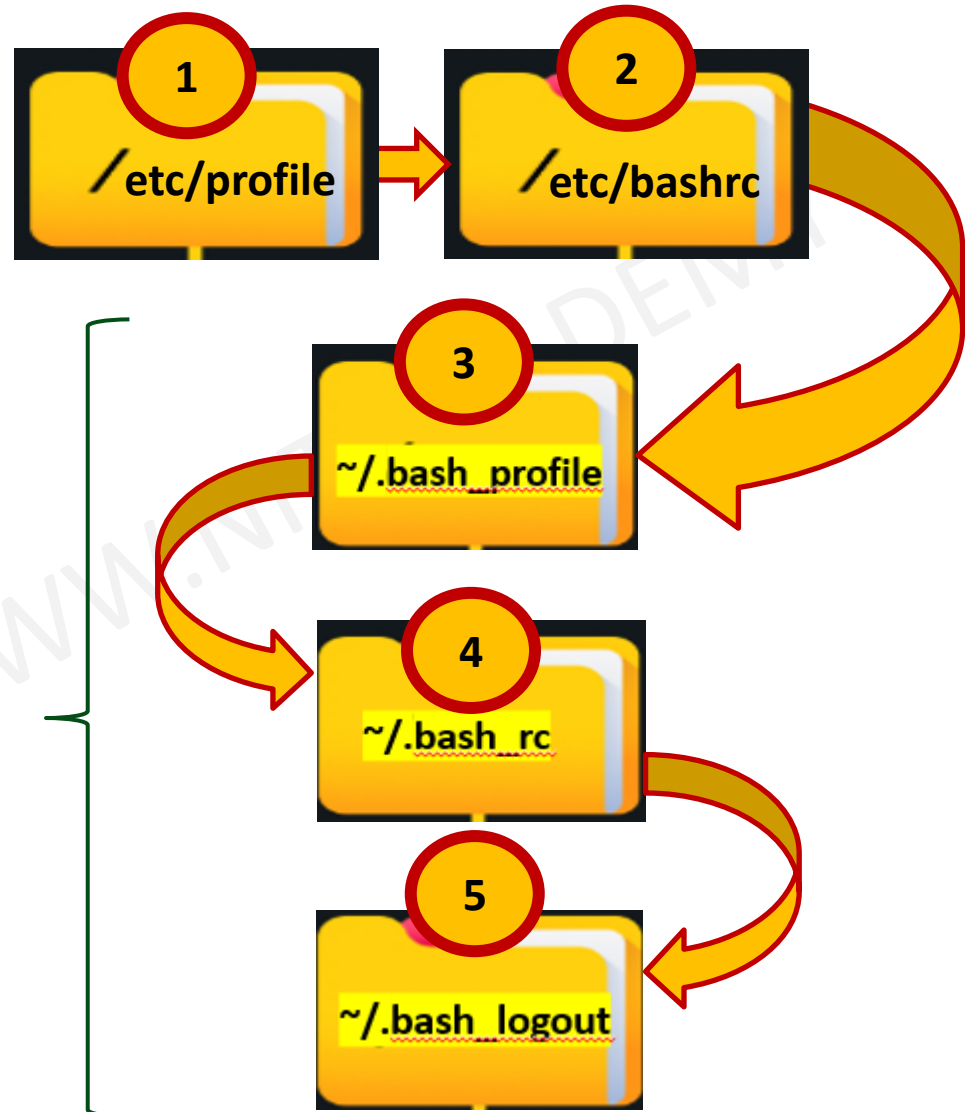
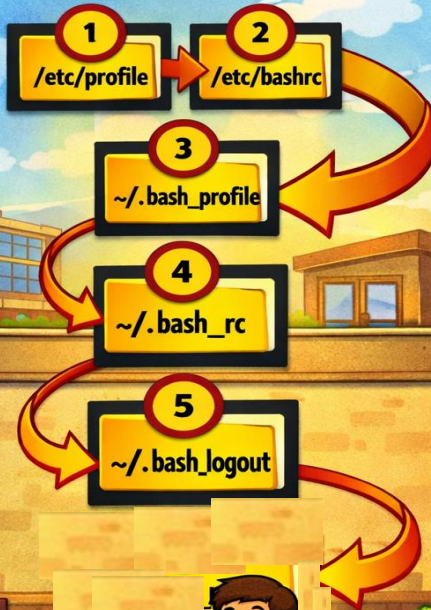
1. User initialization files
2. User can make changes
3. Environmental files found under:

/etc/skel

USER

Initialization scripts set up your environment when you start a Bash shell:

- ✓ System Files (Root) run first
- ✓ User Files run next



USER INITIALIZATION FILES

These are files that each USER can change / modify



Hey as a **User** I can edit this **configuration** file that I can modify and add extra stuff!



Every time I the **User** open a new shell, this file is executed



INITIALIZATION FILES

In Linux, processes don't just appear out of nowhere; with very few exceptions, every process is "forked" (spawned) by an existing **parent process**. Here is exactly what hierarchy means step-by-step:

1. **systemd (The Root Parent)**
2. **sshd (The Child of systemd / Parent of bash)**
3. **bash (The Child of sshd)**
4. Bash reads and **runs initialization files**

Initialization files (like `/etc/profile`, `~/.bash_profile`, and `~/.bashrc`) are put into motion **after bash has already started, but before bash gives you the command prompt**. `sshd` itself doesn't read or execute these files. Instead, `sshd` launches the `bash` process, and one of the very first things `bash` does as it boots up is look for and execute these scripts to set up your environment (like your `PATH`, aliases, and prompt look).

Since, **bash** is the one executing these files, it reads them with the specific characteristics of how it was launched. When `sshd` spawns your shell, it launches it as a

- **login shell (which is why your ps output showed -bash with a hyphen in front of it)**. Because it knows it is a login shell, `bash` follows a very specific checklist of files to execute in this exact order:
 - **/etc/profile** (System-wide settings that apply to all users)
 - **~/.bash_profile** (Your personal login settings—if this doesn't exist, it looks for `~/.bash_login` or `~/.profile`)
 - **~/.bashrc** (Usually called indirectly by your `.bash_profile` to load shell aliases and functions)

If you ever change a setting in `~/.bashrc` and it isn't working, it's because `bash` only processed those files right at that transition state (Step 4). You have to tell `bash` to read them again using

- `source ~/.bashrc`.

5. **SUCCESS:** `bash` presents the **"[root@Linux ~]"** prompt!

LOGIN TO LINUX

Teaching Point

File	Scope
/etc/profile	Global
/etc/bashrc	Global
~/.bash_profile	User Only
~/.bashrc	User Only

CHECK GLOBAL FILES

Global files affect all Users and **only root can modify them**

Let us check if Global Files are available

```
[root@Linux ~]# ls -l /etc/profile
```

```
[root@Linux ~]# ls -l /etc/bashrc
```

How can we view/read these files:

```
[root@Linux ~]# cat /etc/profile
```

```
[root@Linux ~]# cat /etc/bashrc
```

Let us check user files

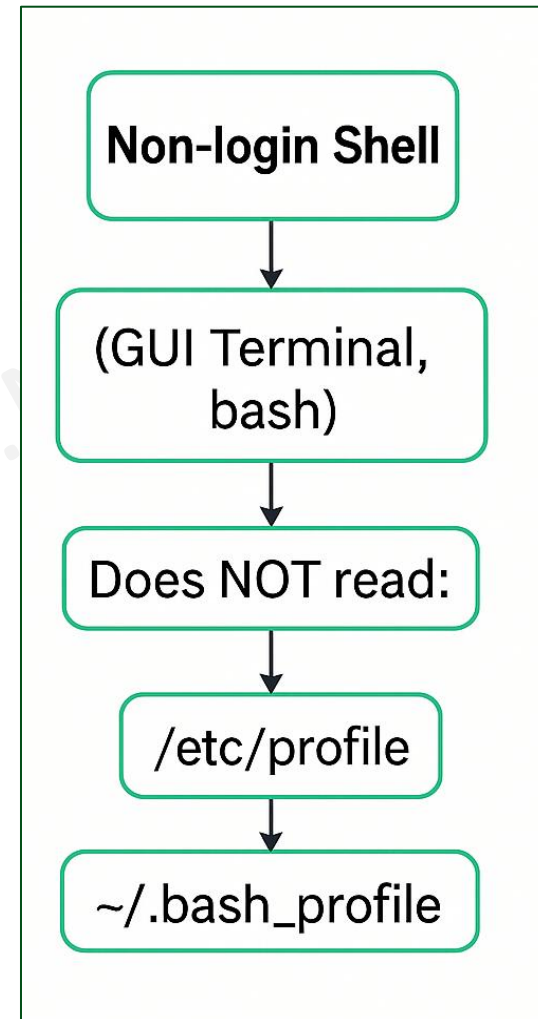
```
[root@Linux etc]# ls -la ~ | grep bash
```

NON-LOGIN SHELL (LINUX)

A no-login interactive shell provides a limited environment.

- An interactive **nologin shell** (often `/sbin/nologin` or `/bin/false`) **does not execute typical shell startup files for interactive sessions.**

Feature	Non-login Shell
Started via	Terminal in GUI, <code>bash</code>
Reads	<code>~/ .bashrc</code> only
Typical use case	Running commands/scripts
Example command	<code>bash</code>



PRACTICE LAB - INITIALIZATION

#Create a New User "john"

```
[root@Linux ~]# useradd john
```

```
[root@Linux ~]# passwd john
```

#Open a duplicate Tab in Guacamole

Passwd:

```
[john@Linux ~]# ip a
```

```
[john@Linux ~]# ssh john@192.168.1.98
```

```
[john@Linux ~]# ps
```

```
[john@Linux ~]# pwd
```

```
[john@john ~]# exit
```

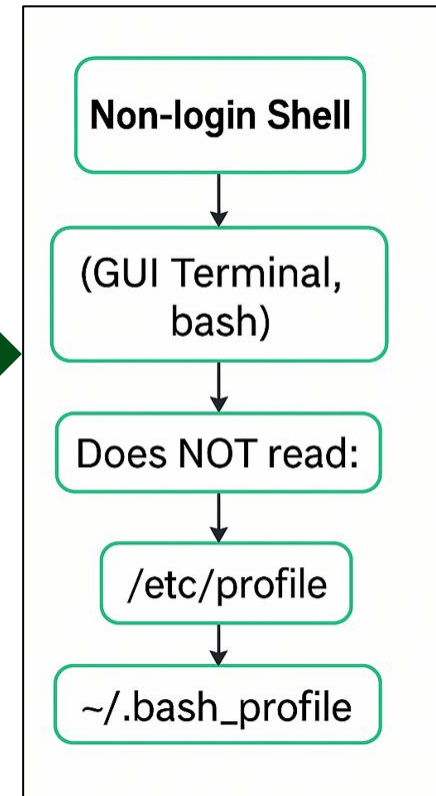
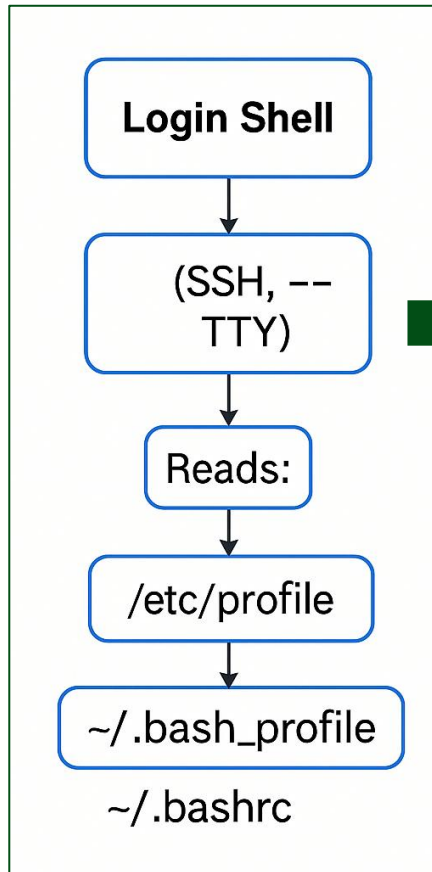
```
[root@Linux ~]# vi /etc/profile
```

add to the bottom echo "/etc/profile"

```
:wq!
```

#Test the message with User John

LOGIN INTERACTIVE



PRACTICE LAB 1



INITIALIZATION FILE SUMMARY

Initializing....
Once in memory

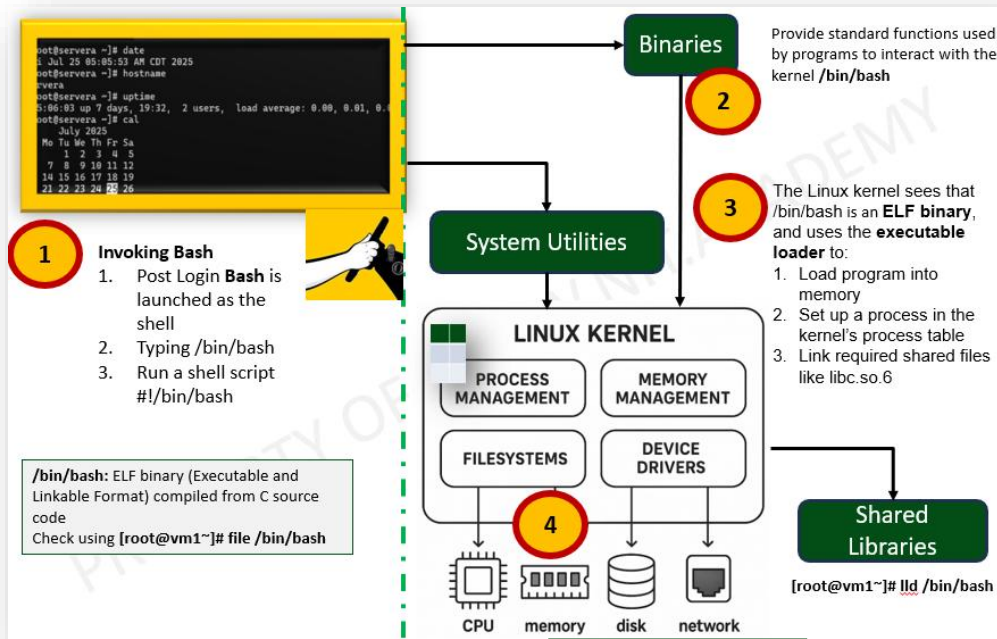
5

[root@Linux~]# cat /etc/profile

What is this file?

1. Maintained by System Admin (and Root)
2. Information in this file **affects all users who log-in**
3. This file sets the shell (environmental) variables like **PATH, USER and HOSTNAME**

Commands
echo \$PATH
echo \$SHELL
echo \$HOST
echo \$USER



[root@Linux~]# cat /etc/bashrc

What is this file?

1. Maintained by System Admin (and Root)
2. Information in this file **affects all users who are already logged-in** and start another session by typing **/bin/bash**
3. When User logs in it is called **no Logon Shell**

REPL – INTERACTIVE LOOP

Bash now enters a loop called a REPL (Read-Eval-Print Loop):

Step	Action
Read	Waits for user to type a command
Eval	Parses and executes the command
Print	Displays output or error
Loop	Waits again for next input

Bash parses the command into:

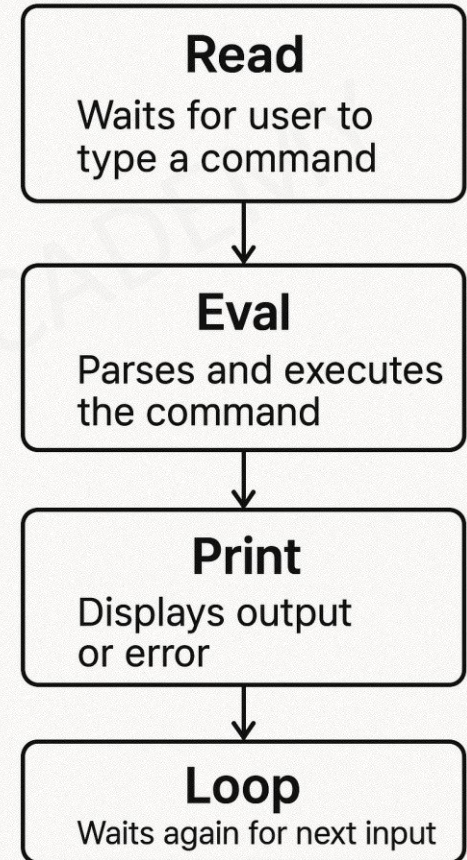
- **ls** (program)
- -l and /etc (arguments)

Then it:

- Locates **ls** using the **\$PATH** variable
- Creates a **child process**
- **Executes /bin/ls** with those arguments
- Waits for the command to finish
- Displays the output

Bash is just a shell (interface) — when you type a command like **uname -r**, it:

- Searches for it in PATH
- Runs it (binary or script)
- Passes arguments if any



WHAT IS PATH IN LINUX?

- When a user types a command like **ls**, **mkdir**, or **systemctl**, the Linux shell doesn't naturally know where those programs live on the hard drive. It relies entirely on the **\$PATH environment variable**.
- **PATH** is just a single long string of directory locations separated by colons (:).
- When a user types a command, the shell reads that string **from left to right**, **checking each folder** one by one until it **finds the executable**. If it hits the end of the string without finding it, it throws the infamous **command not found error**.

The Gold Standard: How it *Should* Work

By clean administrative design, the roles are strictly separated:

1. **/etc/profile** sets the global \$PATH environment variable for everyone because it runs during a **login shell** (like logging in via SSH).
2. **/etc/bashrc** is meant for interactive features (aliases, functions, changing the prompt color/look) for **non-login shells** (like opening a sub-shell or running a remote command script).

Practice Lab – coming up soon!

What **PATH=\$PATH:/getstuff/bin** actually means:

- **PATH=** means we are overwriting the old variable.
- **\$PATH** means "take everything that was already in the path..."
- **:/getstuff/bin** means "...and append a colon followed by our new folder at the very end."

INITIALIZATION PROCESS

Let us Run the following command to see what path been defined by `/etc/profile` - initialization file

```
[root@Linux ~]# echo $PATH
```

```
/root/.local/bin:/root/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin
```

Let us switch to User -> student and see what path has been defined by `/etc/profile` - initialization file

```
[root@Linux ~]# su student
```

```
[student@Linux root]$ echo $PATH
```

```
/home/student/.local/bin:/home/student/bin:/root/.local/bin:/root/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin
```

For Purpose of Learning we will examine:

```
[root@Linux ~]# cd /usr/bin -> systemctl command
```

```
[root@Linux ~]# cd /usr/sbin -> systemctl command
```

Where are these commands hiding

Let us find where in the PATH of `/usr/bin` or `/usr/sbin` we can locate the following => `"ls"`, `"pwd"`, `"touch"`, `"mkdir"`, `"reboot"`, `"restart"`, `"date"`, `"uptime"`,

Dynamic PATH building via `"pathmunge"`

- Prevents Duplication

EXPLANATION

By running **echo \$PATH** as the **root** user, you are looking at the exact **execution blueprint** your shell uses every time **you press Enter**. Here is exactly what you are learning from this specific output:

1. You are Confirmed as the Root User

- Look at the very first two entries: **/root/.local/bin** and **/root/bin**.
- The tilde or literal **/root** directory belongs exclusively to the system administrator.
- If you were logged in as a normal student account (e.g., **student**), those first paths would read **/home/student/.local/bin** and **/home/student/bin**.

2. The Left-to-Right Priority in Action

When you type a command, your shell scans this exact list from left to right, using the colons (:) as split points. The order is:

- **/root/.local/bin** (Personal hidden binaries)
- **/root/bin** (Your custom scripts/executables)
- **/usr/local/sbin** (Custom system admin programs manually installed)
- **/usr/local/bin** (Custom user programs manually installed)
- **/usr/sbin** (Official OS system administration tools like **useradd**, **fdisk**)
- **/usr/bin** (Official OS general user tools like **ls**, **cd**, **grep**)

3. Why the Order Matters: The "Hijack" Protection

- Shell searches **left-to-right** and stops at the *first* match it finds, order dictates security.
- Notice that your custom folders (**/root/bin** and **/usr/local/bin**) come **before** the official OS folders (**/usr/sbin** and **/usr/bin**). This means if you write a custom script, name it **cleanup**, and put it in **/root/bin**, the shell will execute *your* script instead of any system program named **cleanup** first.

4. Why systemctl Works but an App You Just Downloaded Might Not

- If you type **systemctl restart sshd**, the shell looks in **/root/.local/bin** (not there), **/root/bin** (not there)... keeps moving right until it hits **/usr/bin/systemctl**. It finds it, runs it perfectly.
- If you download a new custom DevOps tool or application binary and drop it into a random folder like **/opt/myapp/**, typing **myapp** will fail with **command not found**. Why? Because **/opt/myapp/** is **not** on this list. You would have to use **pathmunge** or manually append it to this string to make it work globally.

PRACTICE LAB

Lab: **Make a Custom Command Global using \$PATH**

Objective: Create a custom script called "hello", witness it fail when run from another directory, and permanently fix it by modifying the .bashrc file

1. Create a custom directory outside of standard paths

```
[root@Linux ~]# mkdir -p /getstuff/bin
```

2. Create a simple shell script inside that directory

```
[root@Linux ~]# echo -e '#!/bin/bash\nnecho "Success! The getstuff application is running!"' > /getstuff/bin/hello
```

3. Make the script executable

```
[root@Linux ~]# chmod +x /getstuff/bin/hello
```

```
[root@Linux ~]# echo $PATH
```

Move to the home directory

```
[root@Linux ~]# cd ~
```

Try to run the new command just by typing its name

```
hello
```


PRACTICE LAB

```
# Append the line to the bottom of ~/.bashrc  
[root@Linux ~]# echo 'PATH=$PATH:/getstuff/bin ; export PATH' >> ~/.bashrc
```

```
# 1. Source the file to apply changes immediately  
[root@Linux ~]# source ~/.bashrc
```

```
# 2. Test the command again from anywhere  
Hello  
[root@Linux ~]# echo $PATH
```

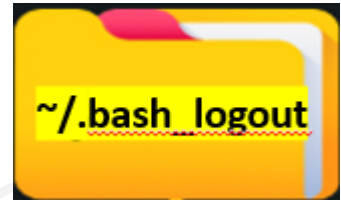
Discussion Question for the Class

*If we created a script named **ls** inside **/getstuff/bin**, would the system run our version of **ls** or the official system version?"* **Answer:** It would run the official version, because our line `PATH=$PATH:/getstuff/bin` put our directory at the *very end* of the list. Bash searches left-to-right and would find the official `/usr/bin/ls` first!

.BASH_LOGOUT FILE...

These are files that each USER can change / modify

```
#vi /home/tom/.bash_logout
```



```
[tom@VM01 ~]$ cat .bash_logout
# ~/.bash_logout

# Clear my terminal Screen on logout
clear

#Remove all temporary files
rm -rf ~/tmp/*

# Unmount any NFS
#umount ~/network_share

#Kill Background processes
kill -9 $(jobs -p)
pkill -u $(whoami)

#Append a Timestamp
#date +"%Y-%m-%d %H:%M:%S Logout by $(whoami)" >> ~/.logout_history
```

```
[root@myserver /]# tree -d /boot (View Directories)
```

```
[root@myserver /]# tree -a -L 2 -h -C
```



HOME DIRECTORY



Login (Home Directory)

- A **home directory**, also called a **login directory**, is the directory on Linux-like operating systems that serves as the repository for a user's personal files, directories, and programs. It is also the directory that a user is first in after logging into the system.
- A **home directory** is created automatically for every ordinary user in the directory called **/home**. Apart from having a home directory to create and store files, **users need an environment** that gives them access to the tools and resources. When a user logs in to a system, the user's work environment is determined by the **initialization files**.
- These **initialization files are defined by the user's startup shell**, which can vary depending on the release. The default initialization files in your home directory enable you to customize your working environment.

FILE SYSTEM CLARIFICATION

Linux Files System is a hierarchy that starts with “File System Root Directory” = “/”

```
[root@Linux ~]# cd /
```

```
[root@Linux /]# pwd  
/
```

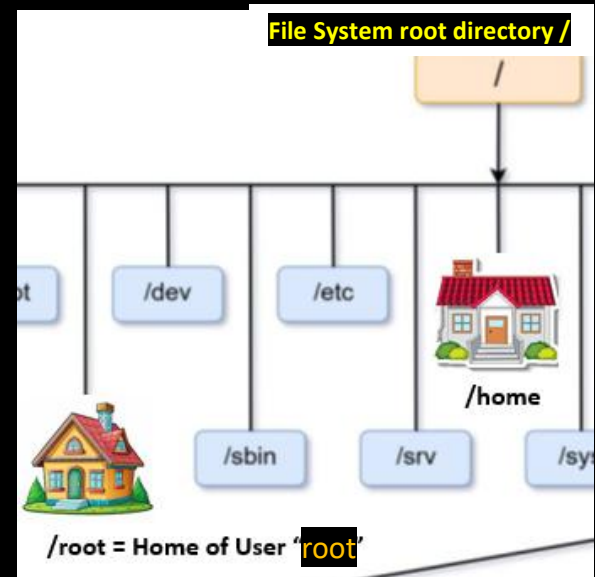
The User **root** is not the same as “Root Directory = /”

```
[root@Linux /]# whoami  
root
```

The home directory of user **root** is called root

```
[root@Linux /]# cd /root
```

```
[root@Linux ~]# pwd  
/root
```



FILE SYSTEM CLARIFICATION

```
# Let us create a Nested DIRECTORY
[root@Linux~ ]# mkdir -p /data/app
[root@ Linux~ ]# ls -l /
[root@ Linux~ ]# cd /data
[root@ Linux~ data ]# pwd
/data

[root@Linux~ data]# mkdir dir1/dir2/dir3/dir4
```

SECURE SHELL (SSH)

Nexus
Office

NOTE: All of these locations are REMOTE LOCATIONS

HOME

- OpenVPN
- VPN
- Tunnel

SSH

DATA CENTER
“on-prem”

Monitor

R620 Dell Poweredge Server



Keyboard

mouse

VPN, Tunnel, OpenVPN

Client

Server

10 - KEYBOARD SHORTCUTS

Date: July-27th-2025



- 1 Ctrl + A => BOL (beginning of line)
- 2 Ctrl + C => Interrupt a process
- 3 Ctrl + D => exit
- 4 Ctrl + E => EOL (end of line)
- 5 Ctrl + L => This clears the screen
- 6 Ctrl + U => Clears line and takes you to BOL (acts like an eraser)
- 7 Ctrl + Z => STOPS a process (BG %1)

8  -----> scroll history UP

9  -----> scroll history DOWN

10  ----> saves time typing

Date: Aug-3rd-2025



USERS - RHCSA EXAM QUESTION

/dev



```
|
|— Devices: `dev`
|
```

- Everything is a file in Linux. Even the devices are viewed as files
- `/dev` stands for "device". Reading from `/dev/sda` is like accessing the hard disk
- Contains special files that represent hardware devices
- Managed dynamically by udev (device manager)

1

#ls /dev

2

#ls -l /dev/sda1

```
[root@VM01 etc]# ls -l /dev/sda1
brw-rw----. 1 root disk 8, 1 Jan 15 09:54 /dev/sda1
[root@VM01 etc]# df -h
```

Filesystem	Size	Used	Avail	Use%	Mounted on
devtmpfs	4.0M	0	4.0M	0%	/dev
tmpfs	385M	0	385M	0%	/dev/shm
tmpfs	154M	4.3M	150M	3%	/run
/dev/mapper/rl_vbox-root	17G	1.6G	16G	10%	/
/dev/sda1	960M	374M	587M	39%	/boot
tmpfs	77M	0	77M	0%	/run/user/1000

Types of Device Files

Type	Description	Examples
Block Devices	Accessed in blocks (random I/O)	/dev/sda, /dev/sdb1
Character Devices	Accessed as a stream (sequential I/O)	/dev/tty, /dev/random

1. When accessing Data in Blocks (Block Devices)

Real-Life Devices:

- Filesystems (like ext4, xfs) use block devices
- Stored under **/dev** as **/dev/sda**, **/dev/xvda**, etc.

Examples:

- Hard drives (**/dev/sda**)
- SSDs
- USB drives

Behavior:

- Data is read or written in **fixed-size chunks**, called **blocks** (typically 512 bytes, 4KB, etc.)
- You **can jump to any block** directly without reading all data before it.

Analogy:

Like reading **pages from a book** — you can go to **page 100** directly without reading pages 1–99.

2. When accessing Data as a stream (Character Devices)

Real-Life Devices:

- Serial consoles, pipes, sensors
- Found as **/dev/tty***, **/dev/zero**, **/dev/random**

Examples:

- Keyboard (**/dev/tty**)
- Mouse (**/dev/input/mouse0**)
- Serial ports (**/dev/ttyS0**)

Behavior:

- Data is accessed as a **continuous flow** (byte-by-byte or character-by-character).
- You **cannot seek** or skip ahead — you read data **in the order it comes in**.

Analogy:

Like **listening to a radio** — you hear sounds as they come; you can't skip ahead.

Types of TTYs in Your Screenshot:

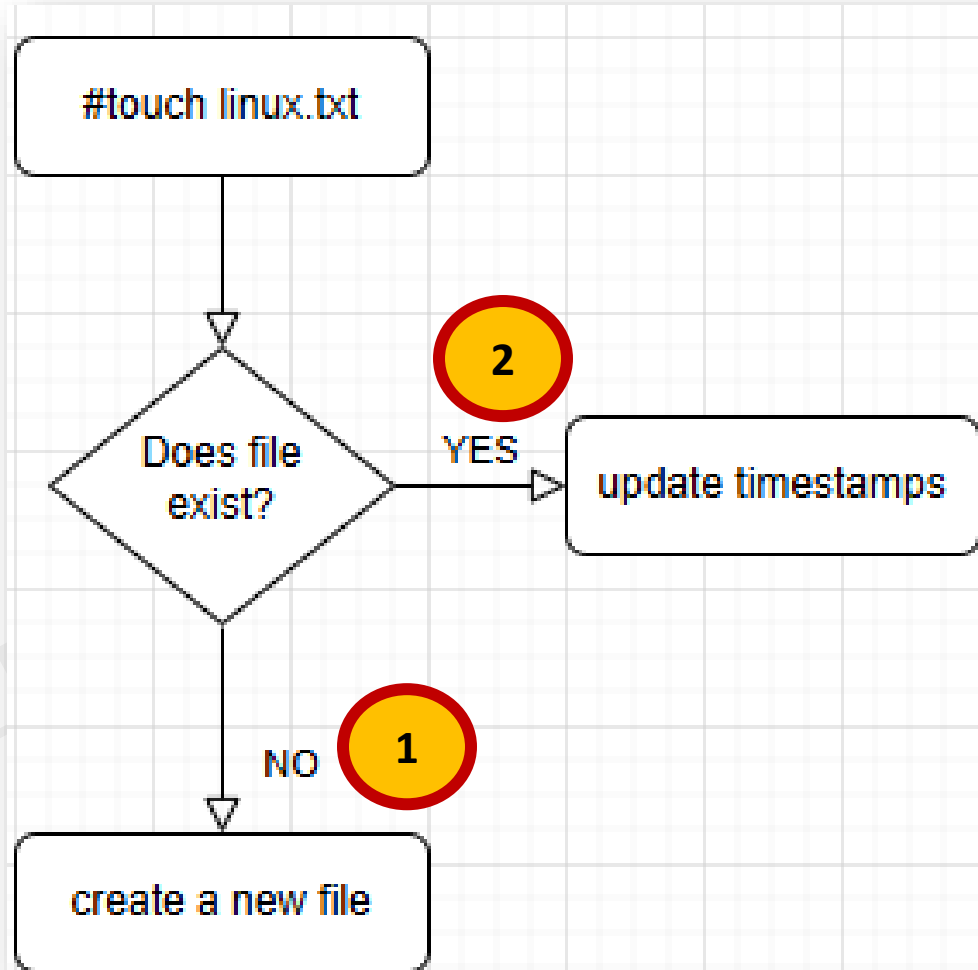
TTY	Explanation
tty1	A virtual console directly on the machine
pts/0	A pseudo-terminal slave used for remote or terminal emulator sessions , like SSH

```
[root@myserver ~]# w
01:03:26 up 15 min, 2 users, load average: 0.03, 0.07, 0.09
USER  TTY      LOGIN@  IDLE   JCPU   PCPU   WHAT
root  tty1     00:48   14:30   0.01s  0.01s  -bash
root  pts/0    01:01   0.00s  0.05s  0.02s  w
```

Command	What it does	Terminal 1	Terminal 2
tty	Shows which terminal (e.g., /dev/pts/0)	[root@Linux ~]# tty /dev/pts/0	[root@Linux ~]# tty /dev/pts/1
w / who	Shows all logged-in users with TTY	[root@Linux ~]# w 01:46:22 up 57 min, 4 users, load average: 0.07, 0.06, 0.05 USER TTY LOGIN@ IDLE JCPU PCPU WHAT root tty1 00:48 38:06 0.26s 0.26s -bash root pts/0 01:01 5:49 0.05s 0.05s -bash root pts/1 01:12 0.00s 0.30s 0.13s w	
ls /dev/pts	Lists all pseudo terminals	[root@myserver ~]# ls /dev/pts 0 1 2 ptmx	
echo "msg" > /dev/pts/X	Sends a message to another terminal	echo "Hello from pts/0" > /dev/pts/1	

TOUCH COMMAND

#touch



TOUCH COMMAND

1

File does not exist

STEP 1: List what is under vm2 directory

```
[vm2@System-Admin ~]$ ls
a.txt  b.txt
```

STEP 2: Create a new file called 'Linux.txt' (Does not exist)

```
[vm2@System-Admin ~]$ touch linux.txt
[vm2@System-Admin ~]$ ls
a.txt  b.txt  linux.txt
[vm2@System-Admin ~]$ ls -l linux.txt
-rw-r--r--. 1 vm2 vm2 0 Jan 11 22:23 linux.txt
```

Step 3: Check STATS of 'Linux.txt' and Check Timestamps (all the same)

```
[vm2@System-Admin ~]$ stat linux.txt
  File: linux.txt
  Size: 0                Blocks: 0          IO Block: 4096   regular empty file
Device: fd00h/64768d    Inode: 50833963  Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/    vm2)   Gid: ( 1000/    vm2)
Context: unconfined_u:object_r:user_home_t:s0
Access: 2025-01-11 22:23:33.023218089 -0600
Modify: 2025-01-11 22:23:33.023218089 -0600
Change: 2025-01-11 22:23:33.023218089 -0600
 Birth: 2025-01-11 22:23:33.023218089 -0600
```

TOUCH COMMAND

2

File is existing

Step 1:

```
[vm2@System-Admin ~]$ ls
a.txt  b.txt  linux.txt
```

Step 2:

Using 'touch' on 'linux.txt' => existing file => Timestamps are UPDATED

```
[vm2@System-Admin ~]$ touch linux.txt
```

```
[vm2@System-Admin ~]$ stat linux.txt
```

File: linux.txt

Size: 0 Blocks: 0 IO Block: 4096 regular empty file

Device: fd00h/64768d Inode: 50833963 Links: 1

Access: (0644/-rw-r--r--) Uid: (1000/ vm2) Gid: (1000/ vm2)

Context: unconfined_u:object_r:user_home_t:s0

Access: 2025-01-11 22:24:40.233137428 -0600

Modify: 2025-01-11 22:24:40.233137428 -0600

Change: 2025-01-11 22:24:40.233137428 -0600

Birth: 2025-01-11 22:23:33.023218089 -0600

TOUCH COMMAND – CHANGING DATE AND TIME

Setting Access Time to a New Date, Time and Secs:

```
[nitadmin2@VM01 ~]$ touch -a -t 202301020000.00 linux.txt
```

```
[nitadmin2@VM01 ~]$ stat linux.txt
```

File: linux.txt

Size: 0 Blocks: 0 IO Block: 4096 regular empty file

Device: fd00h/64768d Inode: 50835693 Links: 1

Access: (0644/-rw-r--r--) Uid: (1000/nitadmin2) Gid: (1000/nitadmin2)

Context: unconfined_u:object_r:user_home_t:s0

Access: 2023-01-02 00:00:00.000000000 -0600

Modify: 2024-12-31 22:00:00.000000000 -0600

Change: 2025-01-13 17:16:11.956748247 -0600

Birth: 2025-01-13 16:53:05.400443642 -0600

Setting Modify Time to a New Date, Time and Secs:

```
[nitadmin2@VM01 ~]$ touch -m -t 202412151100.45 linux.txt
```

```
[nitadmin2@VM01 ~]$ stat linux.txt
```

File: linux.txt

Size: 0 Blocks: 0 IO Block: 4096 regular empty file

Device: fd00h/64768d Inode: 50835693 Links: 1

Access: (0644/-rw-r--r--) Uid: (1000/nitadmin2) Gid: (1000/nitadmin2)

Context: unconfined_u:object_r:user_home_t:s0

Access: 2023-01-02 00:00:00.000000000 -0600

Modify: 2024-12-15 11:00:45.000000000 -0600

Change: 2025-01-13 17:19:04.702552836 -0600

Birth: 2025-01-13 16:53:05.400443642 -0600

TOUCH COMMAND

Synchronizing atime & mtime

Before

```
[nitadmin2@VM01 ~]$ stat linux.txt
File: linux.txt
Size: 0                Blocks: 0                IO Block: 4096    regular empty file
Device: fd00h/64768d   Inode: 50835693       Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/nitadmin2)   Gid: ( 1000/nitadmin2)
Context: unconfined_u:object_r:user_home_t:s0
Access: 2023-01-02 00:00:00.000000000 -0600
Modify: 2024-12-15 11:00:45.000000000 -0600
Change: 2025-01-13 17:25:33.788199032 -0600
Birth: 2025-01-13 16:53:05.400443642 -0600
```

```
[nitadmin2@VM01 ~]$ touch -a linux.txt
```

```
[nitadmin2@VM01 ~]$ stat linux.txt
```

```
File: linux.txt
Size: 0                Blocks: 0                IO Block: 4096    regular empty file
Device: fd00h/64768d   Inode: 50835693       Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/nitadmin2)   Gid: ( 1000/nitadmin2)
Context: unconfined_u:object_r:user_home_t:s0
Access: 2025-01-13 17:26:41.131361381 -0600
Modify: 2024-12-15 11:00:45.000000000 -0600
Change: 2025-01-13 17:26:41.131361381 -0600
Birth: 2025-01-13 16:53:05.400443642 -0600
```

```
[nitadmin2@VM01 ~]$ touch -m linux.txt
```

```
[nitadmin2@VM01 ~]$ stat linux.txt
```

```
File: linux.txt
Size: 0                Blocks: 0                IO Block: 4096    regular empty file
Device: fd00h/64768d   Inode: 50835693       Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/nitadmin2)   Gid: ( 1000/nitadmin2)
Context: unconfined_u:object_r:user_home_t:s0
Access: 2025-01-13 17:26:41.131361381 -0600
Modify: 2025-01-13 17:27:02.823301904 -0600
Change: 2025-01-13 17:27:02.823301904 -0600
Birth: 2025-01-13 16:53:05.400443642 -0600
```

Access time
synchronized

Modify time
synchronized

STAT VS LS COMMAND

STEP 1: Check all Time Stamps using STAT Command

```
[nitadmin2@VM01 ~]$ stat linux.txt
File: linux.txt
Size: 0                      Blocks: 0                      IO Block: 4096   regular empty file
Device: fd00h/64768d        Inode: 50835693           Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/nitadmin2)   Gid: ( 1000/nitadmin2)
Context: unconfined_u:object_r:user_home_t:s0
Access: 2023-11-29 22:00:00.000000000 -0600
Modify: 2024-12-15 01:00:45.000000000 -0600
Change: 2025-01-13 19:55:19.268544270 -0600
Birth: 2025-01-13 16:53:05.400443642 -0600
```

STEP 2: Check a-time (access time) using `ls -ltu` command

```
[nitadmin2@VM01 ~]$ ls -ltu linux.txt
-rw-r--r--. 1 nitadmin2 nitadmin2 0 Nov 29 2023 linux.txt
```

STEP 3: Check m-time (modify time) using `ls -lt` command

```
[nitadmin2@VM01 ~]$ ls -lt linux.txt
-rw-r--r--. 1 nitadmin2 nitadmin2 0 Dec 15 01:00 linux.txt
```

TIME & DATE – FULL VIEW

#ls -l --full-time

STEP 1: Check all Timestamps

```
[nitadmin2@VM01 ~]$ stat linux.txt
```

```
File: linux.txt
Size: 0          Blocks: 0          IO Block: 4096   regular empty file
Device: fd00h/64768d    Inode: 50835693   Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/nitadmin2)   Gid: ( 1000/nitadmin2)
Context: unconfined_u:object_r:user_home_t:s0
Access: 2023-11-29 22:00:00.000000000 -0600
Modify: 2024-12-15 01:00:45.000000000 -0600
Change: 2025-01-13 19:55:19.268544270 -0600
Birth: 2025-01-13 16:53:05.400443642 -0600
```

STEP 2: Check Access Time using ls command

```
[nitadmin2@VM01 ~]$ ls -ltu --full-time linux.txt
```

```
-rw-r--r--. 1 nitadmin2 nitadmin2 0 2023-11-29 22:00:00.000000000 -0600 linux.txt
```

STEP 3: Check Modify Time using 'ls' command

```
[nitadmin2@VM01 ~]$ ls -lt --full-time linux.txt
```

```
-rw-r--r--. 1 nitadmin2 nitadmin2 0 2024-12-15 01:00:45.000000000 -0600 linux.txt
```

THANK YOU
Questions?